Energy-Efficient Smart Home Automation: A Raspberry Pi-Based Approach to Reducing IoT Energy Consumption

Christine Bonoan, Jaden Bowers, Brandon Darghali, Enrique Medrano, Roberto Moreno
Department of Computer & Electrical Engineering & Computer Science
[redacted]@csub.edu

Abstract – *Smart home IoT devices aim to reduce energy consumption compared to a home with traditional appliances. However, inefficiencies in automation, sensor accuracy, and standby power can contribute to excess usage. Existing research focuses on energy monitoring, predefined smart home models, and creating a separate home automation system, but they lack dynamic optimization strategies within the devices. This paper introduces an approach to demonstrate energy efficiency through smart lighting systems by implementing software and hardware modifications using a Raspberry Pi. Improved automation logic, additional and modified sensor behavior, and real-time energy monitoring aim to help minimize unnecessary energy usage. The proposed method optimizes existing devices without need for replacements, benefiting smart home owners and making it feasible for real-world applications.*

Keywords: Energy optimization, Energy efficiency, Energy consumption, Smart homes, Internet of Things, Smart meter, Home automation

1 Introduction

The rapid expansion of the Internet of Things (IoT) has encouraged the transformation of traditional living spaces into modern smart homes. IoT smart home devices rely on network connectivity and mobile applications or software [1], giving the user more control over usage. The integration of connected devices for optimized energy use has resulted in lower energy bill costs, environmental sustainability, and increased human quality of life [2], [3]. Despite their intended benefits, smart devices yet contribute to excessive energy consumption due to inefficiencies in automation logic, sensor accuracy, and standby power consumption.

Concerns of financial burdens and sustainability arise with the use of traditional home appliances, leading to a demand for smart home solutions. [1] presents a table comparing energy expenditure between a home without smart devices in use and the same home with the devices activated. Although energy consumption is reduced by about 16%, there is no question whether these smart home devices are fully optimized to prevent inefficiencies.

While IoT devices in the home promise increased energy management, their default automation settings often result in excess energy consumption. They introduce challenges in managing this consumption effectively, such as the issue of phantom power, or standby consumption [4]. For instance, smart lighting systems are utilized with the effort of enhancing user comfort and reducing energy waste [5], [6]. Smart lighting systems are a culprit to phantom power as the standby energy of dozens of smart light bulbs in a home can add in excess and "surpass the potential energy waste of misusing legacy devices" [4]. They also create an issue of unnecessary activation. These lighting systems may activate even when the user does not need it due to preset scheduling or inaccuracies in their sensor technologies. Such issues can contribute to unexpected increases in energy bills and negate the intention of saving energy.

It is crucial to address energy inefficiencies to maximize energy conservation and optimize the performance of smart home devices. Current research has focused on shutting down devices through machine learning prediction [4], user awareness of devices' energy consumption [7], setting fixed energy limits per household [8], predefining smart home models [9], heavier focus on communication protocols [10], implementing computer vision through thermal sensors [11], using similar sensors [12], and smart meter monitoring [13], [14], [15]. However, they do not directly address dynamically optimizing device behavior through software and hardware modifications within each device.

This paper presents an integrated approach to minimizing energy consumption in smart home IoT devices by implementing both software and hardware-based optimizations through improved automation logic, additional and modified sensor behavior, and real-time energy meter readings with the use of a Raspberry Pi. The study focuses on portable and commonly used smart home devices, such as smart lighting systems, making it feasible for real-world implementation and demonstration. The scope of this research is limited to enhancing the energy efficiency of existing IoT devices without replacing them.

The rest of the paper is organized as follows. Section II discusses related work on smart home energy optimization strategies. Section III presents our proposed methodology and design process. Section IV presents the results and energy saving analysis. Section V concludes the paper.

2 Related Work

The integration of IoT devices in homes has led to an increased interest in energy efficiency and optimization within smart homes. Prior studies have primarily focused on the use of machine learning and computer vision, energy monitoring and scheduling, proposed smart home models and fixed limits, and individual device control.

In [4], wasted energy consumption of IoT devices in standby mode is addressed by predicting usage times based on usage history and turning on the associated device when predicted necessary. Similarly, Goel, Bhatia, and Rana [10] address the importance of low-power control systems by using an Arduino Uno as a controller, a Bluetooth module, and a relay module to enter a low-power sleep mode when not in use and optimizing code to limit unnecessary device operations. However, no hardware adjustments are made to mitigate wasted energy consumption.

Another field of artificial intelligence, computer vision, is used in [11] with a thermal camera and a normal camera to accurately predict presence in a room and activate a smart light accordingly. This implements another sensor, like our proposed method, but differs with its training and learning and does not combine multi-sensor input or monitor real-time energy feedback.

[7] presents an automation system that monitors devices and relays the status and information, such as energy consumption, of those devices. From there, the user can control the devices based on what the information reveals. The research relies heavily on static configurations such as scheduling rather than adapting to real-time environmental data. It relies more on monitoring and user interaction through an interface, whereas our proposed method is more responsive to context triggers and granular energy data. The research in [8] emphasizes appliance scheduling to follow a daily maximum energy consumption limit using a proposed Daily Maximum Energy Scheduling (DMES) technique. However, this also does not adapt to contextual situations.

In [9], a smart home model equipped with different IoT devices is proposed and the power consumption of each device is measured as well. Although the proposed would be an efficient way to reduce energy consumption, the simulation must be planned beforehand, and the insight cannot exactly replicate real environments. Simulated systems do not account for unpredictable user behavior, inaccurate sensors, or fluctuation conditions within the home.

Research in [13], [14], and [15] all implement smart meter monitoring that is then displayed to the user. In [13] and [14], energy consumption management is done through monitoring energy usage data collected by smart meter system. The data is displayed in an application or cloud to be analyzed by the user. In [15], a mobile application is also created to display data from the created smart meter system. The smart meter can be controlled and managed by the user through the mobile application, and utility companies can "monitor and administer" the smart system through a developed website [15]. While all three systems support energy monitoring for user awareness, they primarily emphasize data collection and visualization rather than implementing real-time, sensor-based automation to actively reduce energy waste.

Current research on smart home energy management emphasizes smart monitoring systems, simulation models, and device control through communication protocols, but they lack an approach that integrates real-time sensor input with automation logic for dynamic optimization. Our research addresses this gap by demonstrating how hardware and software modifications—using Raspberry Pi, smart plugs, and environmental sensors—can enhance automation and reduce energy waste without requiring device replacement. There is one [12] that is similar to our proposed method, however, which implements a passive infrared (PIR) and a light dependent resistor (LDR), but it primarily focuses on basic lighting scenarios and is limited to the PIR motion sensor. Its limitations are prone to more false negatives and lower precision, which will be addressed in our proposed method by adding three sensors, refining automation logic, and implementing real-time energy monitoring. It would also be a more responsive and efficient energy management approach. This work provides a practical, scalable, and more effective solution for improving energy efficiency in existing smart home setups.

3 Methodology and Implementation

Our proposed solution involves an efficient and scalable smart lighting system while integrating various sensors and smart devices to design an energy-efficient smart lighting system. It utilizes both software automation and hardware-based sensing. The system architecture is designed with a Raspberry Pi 4 Model B that acts as a central controller. It connects with the sensors and communicates with Zigbee-compatible devices using the Zigbee2MQTT bridge and the Mosquitto Message Queuing Telemetry Transport (MQTT) broker for communication.

3.1 Setup and Integration

The hardware components include a Zigbee supported Philips Hue Smart LED bulb, THIRDREALITY Zigbee Smart Plug, Sonoff Zigbee 3.0 USB Dongle Plus (ZBDongle-E), PIR motion sensor, human micro-motion millimeter-wave (mmWave) sensor, BH1750 light lux sensor, and supporting materials such as a breadboard, Ethernet cable, heatsink fan plus heatsinks, and a light socket to plug adapter. In this setup (Fig. 1), the USB dongle is flashed with the Ember firmware and connected to the Raspberry Pi via USB. This allows the Zigbee2MQTT to discover and control both the smart light bulb and the smart plug. The smart plug, used for real-time energy consumption monitoring, is plugged into a wall outlet (not pictured) and powers the smart bulb.

The PIR, mmWave, and light lux sensors are connected to the Raspberry Pi through GPIO pins and I2C interfaces using a

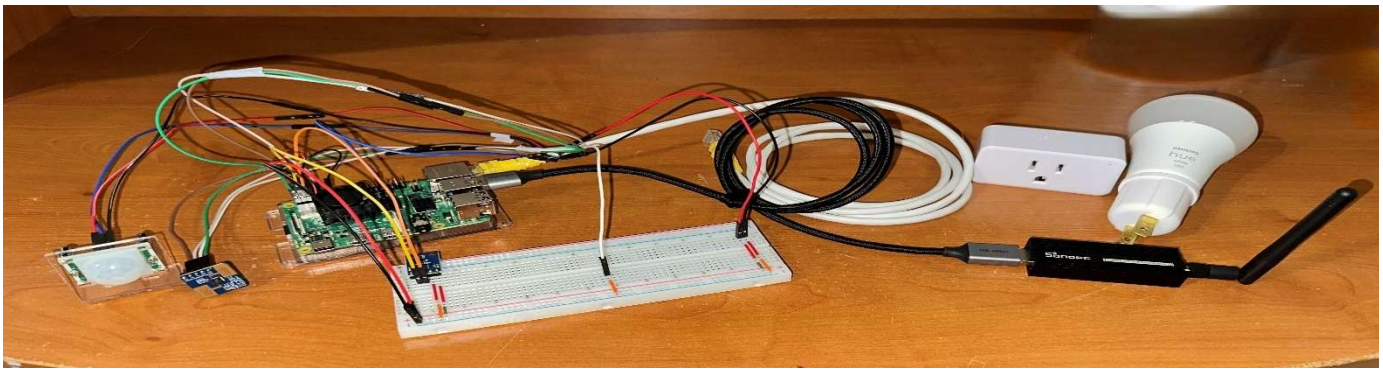

Fig. 1. Smart Lighting System Hardware Setup

breadboard. The entire setup is placed in an approximately 10 ft x 12 ft bedroom, with the sensors placed approximately 4 ft from the window that faces west and gets sunlight from 3 PM to around 7-8 PM. The PIR and mmWave sensors are placed near the bedroom door, positioned on a shelf approximately 3 ft above the ground and the mmWave oriented toward the center and opposite end of the room. According to their specifications, the PIR motion sensor has a detection cone angle of less than 100 degrees, which makes it effective for detection motion directly in front of the sensor (i.e., walking past the sensor). It is positioned to monitor the bedroom door where motion is most expected. The mmWave sensor detects motion up to 10 meters and micro-motion up to 6 meters. This is ideal for tracking both large and subtle movements and presence in the room. The light lux sensor is placed on the breadboard facing upward and is exposed to ambient brightness levels throughout the day.

3.2 Communication and Control Logic

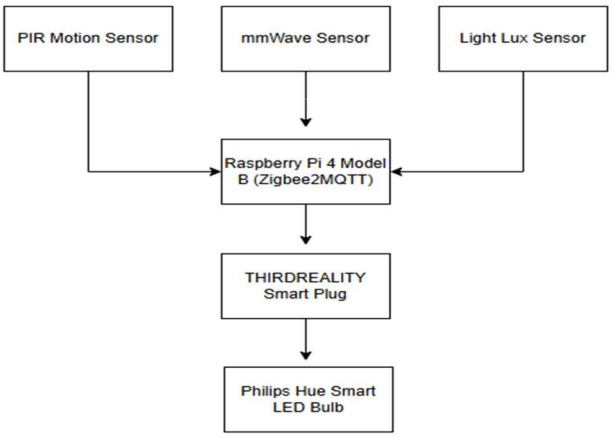

Fig. 2. Detection diagram between sensors, Raspberry Pi, smart plug, and smart light bulb

Zigbee2MQTT is used as a service on the Raspberry Pi and facilitates MQTT communication with the Zigbee-supported bulb and smart plug. Custom scripts are written with Python version 3.11.2 to handle sensor logic and logging, and according to the detection diagram (Fig. 2), the devices listen to the MQTT messages from the scripts that send control commands based on the sensor logic.

The control logic is designed as described:

- If the ambient light (lux) falls below a predetermined threshold and either the PIR or mmWave sensors detect motion or presence, turn on the light bulb and adjust the brightness accordingly.
- If no presence is detected for a defined period (10 seconds), turn off the bulb to save energy.
- Smart plug continuously reports real-time power usage and logs the data, which is then analyzed to evaluate energy consumption patterns.

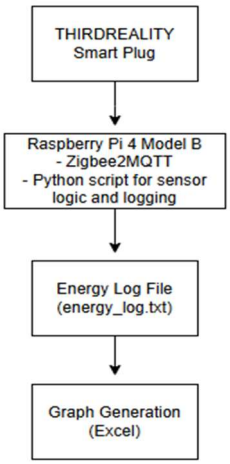

Fig. 3: Data gathering diagram between smart plug, Raspberry Pi implementing Zigbee2MQTT and Python scripts, and sent to log file to be tabled and graphed for analysis.

The data gathered (Fig. 3) from the smart plug is logged into a .txt file from lines in the script. We initially had it set up to print in the console for real-time reading, but to keep the console from being cluttered with different types of lines necessary testing purposes, we logged the real-time energy readings into the .txt file.

3.3 Network and Power Configuration

The Raspberry Pi 4 Model B is connected to a laptop via Ethernet during testing. The bulb is screwed into the light socket to plug adapter, which is then plugged into the smart plug. The sensors are powered through the Pi's GPIO pins. The smart plug controls power delivery to the bulb and provides

energy usage feedback through values of power, voltage, and current. The total energy consumed is calculated through these values, necessary for studying the system's energy-saving effectiveness.

3.4 Source Code

We went with the approach of using modules to structure the code logically and make it easier to understand and maintain. We also wanted to reduce code duplication and prevent having one large, exhaustive piece of source code. Modules were used for the light bulb control (bulb_control.py) (Fig. 4) and each sensor (pir_sensor.py, mmwave_sensor.py, light_sensor.py) (Fig. 5, Fig. 6, Fig, 7). We then created a main3.py, main4.py, and main5.py script (number signifying the testing day) to import the modules and utilize the functions within them. Specific logic for entering, exiting, and staying in the room were done inside the main scripts.

Algorithm 1 Turn Smart Bulb ON

```
procedure TURN_ON
    Publish MQTT message:
        Topic: zigbee2mqtt/hue-bulb/set
        Host: localhost
        Message: {"state": "ON"}
    Output "Bulb ON"
end procedure
```

Algorithm 2 Turn Smart Bulb OFF

```
procedure TURN_OFF
    Publish MQTT message:
        Topic: zigbee2mqtt/hue-bulb/set
        Host: localhost
        Message: {"state": "OFF"}
    Output "Bulb OFF"
end procedure
```

Fig. 4: Pseudocode for bulb control logic (bulb_control.py).

Algorithm 1 Initialize PIR Motion Sensor

```
Define PIR sensor GPIO pin: PIR_PIN ← 12
Set GPIO mode to BCM
Configure PIR_PIN as input
```

Algorithm 2 Check for Motion Using PIR Sensor

```
function MOTION_DETECTED
    return True if input from PIR_PIN is HIGH (1)
    return False otherwise
end function
```

Fig. 5: Pseudocode for PIR motion sensor (pir_sensor.py).

Algorithm 1 Initialize mmWave Sensor Communication

```
Open serial connection on port /dev/ttyS0
Set baud rate to 115200 and timeout to 1 second
Send initialization command:
    Hex: FDFCFBFA080012000000640000004030201
```

Algorithm 2 Read Distance from mmWave Sensor

```
function GET_MMWAVE_DISTANCE
    Read a line from serial and decode as UTF-8
    if line is not empty then
        Output raw data
    end if
    if line starts with "Range" then
        Extract numeric value following "Range"
        Convert to integer as distance_cm
        Output distance
        return distance_cm
    end if
    return None                          ▷ If data is invalid or not parseable
end function
```

Fig. 6: Pseudocode for mmWave sensor (mmwave_sensor.py).

Algorithm 1 Initialize BH1750 Light Sensor

```
DEVICE ← 0x23                            ▷ BH1750 I2C address
POWER_ON ← 0x01
RESET ← 0x07
CONT_H_RES_MODE ← 0x10
Use I²C bus 1
Send command: POWER_ON
Send command: RESET
Send command: CONT_H_RES_MODE
```

Algorithm 2 Read Lux from BH1750 Sensor

```
function READ_LUX
    data ← read 2 bytes from DEVICE using CONT_H_RES_MODE
    light_level ← (data[0] << 8) | data[1]
    return light_level ÷ 1.2             ▷ Convert to lux
end function
```

Algorithm 3 Calculate Brightness Based on Ambient Light

```
function CALCULATE_BRIGHTNESS(lux, min_lux, max_lux, min_brightness)
    if lux = None then
        return 0
    else if lux ≥ max_lux then
        return 0                         ▷ Room is already bright
    else if lux ≤ min_lux then
        return 254                       ▷ Room is dark — full brightness
    else
        scale ← (max_lux − lux) ÷ (max_lux − min_lux)
        brightness ← scale × (254 − min_brightness) + min_brightness
        return integer part of brightness
    end if
end function
```

Fig. 7: Pseudocode for light lux sensor (light_sensor.py).

The main3.py (Fig. 8) script imported the bulb control and PIR modules and was used in Day 3's testing. Bulb control, PIR, and mmWave modules were imported in the main4.py (Fig. 9) script and used on Day 4. Lastly, the bulb control, PIR, mmWave, and light sensor modules were imported in main5.py (Fig. 10) and used on Day 5. Pseudocode snippets are shown below:

Algorithm 1 Initialize PIR Motion Sensor

```
PIR_PIN ← 12
Set GPIO mode to BCM
Configure PIR_PIN as input
```

Algorithm 2 Motion Detection Function

```
function MOTION_DETECTED
    if GPIO.input(PIR_PIN) = 1 then
        return True
    else
        return False
    end if
end function
```

Fig. 8: Pseudocode for Day 3 testing with PIR sensor only (main3.py).

Algorithm 1 Main Control Loop for Presence-Based Lighting

```
light_on ← False
startup_checked ← False
last_motion_time, last_distance_time, exit_time ← 0
WINDOW_EXIT ← 3 seconds
WINDOW_ENTRY ← 5 seconds
DISTANCE_THRESHOLD ← 80 cm
EXIT_COOLDOWN ← 5 seconds
while True do
    now ← current time
    motion ← MOTION_DETECTED
    distance_cm ← GET_MMWAVE_DISTANCE
    if distance_cm = None then
        continue
    end if
                                    ▷ Suppress mmWave readings for cooldown period after exit
    if light_on = False and now − exit_time < EXIT_COOLDOWN then
        last_distance ← 0
        last_distance_time ← 0
    else
        last_distance ← distance_cm
        last_distance_time ← now
    end if
    if motion = True then
        last_motion_time ← now
    end if
                                    ▷ Startup: user already in room (far away)
    if startup_checked = False and light_on = False and distance_cm > DISTANCE_THRESHOLD then
        TURN_ON
        light_on ← True, startup_checked ← True
        continue
    end if
                                    ▷ Exit: motion just before short-range detection
    if light_on = True and 0 < last_distance_time − last_motion_time < WINDOW_EXIT and last_distance < DISTANCE_THRESHOLD then
        TURN_OFF
        light_on ← False, exit_time ← now
        continue
    end if
                                    ▷ Entry: motion then distant object within 5s
    if light_on = False and now − exit_time > EXIT_COOLDOWN and 0 < last_distance_time − last_motion_time < WINDOW_ENTRY and last_distance > DISTANCE_THRESHOLD then
        TURN_ON
        light_on ← True
        continue
    end if
                                    ▷ Block entry logic during cooldown
    if light_on = False and now − exit_time ≤ EXIT_COOLDOWN then
                                    ▷ Do nothing
    end if
                                    ▷ Confirm presence if user is still far away
    if light_on = True and distance_cm > DISTANCE_THRESHOLD then
                                    ▷ Do nothing — user is still in room
    end if
end while
```

Fig. 9: Pseudocode for Day 4 testing with PIR and mmWave sensors (main4.py).

Algorithm 1 Full System Integration Loop (PIR, mmWave, Lux, Bulb)

```
Initialize global lux ← 0.0
Start background thread to periodically update lux using Read_Lux() every 0.15s
light_on ← False
startup_checked ← False
last_motion_time, last_distance_time, exit_time ← 0
WINDOW_EXIT ← 3, WINDOW_ENTRY ← 5
DISTANCE_THRESHOLD ← 80
EXIT_COOLDOWN ← 5
while True do
    now ← current time
    motion ← MOTION_DETECTED
    distance_cm ← GET_MMWAVE_DISTANCE
    if distance_cm = None then
        continue
    end if
    if light_on = False and now − exit_time < EXIT_COOLDOWN then
        Suppress mmWave
        last_distance, last_distance_time ← 0
    else
        last_distance ← distance_cm
        last_distance_time ← now
    end if
    if motion = True then
        last_motion_time ← now
    end if
                                    ▷ Startup: already in room
    if startup_checked = False and light_on = False and distance_cm > DISTANCE_THRESHOLD then
        brightness ← CALCULATE_BRIGHTNESS(lux)
        if brightness > 0 then
            SET_BULB_BRIGHTNESS(brightness)
            light_on ← True
        end if
        startup_checked ← True
        continue
    end if
                                    ▷ Exit logic
    if light_on = True and 0 < last_distance_time − last_motion_time < WINDOW_EXIT and last_distance < DISTANCE_THRESHOLD then
        TURN_OFF
        light_on ← False
        exit_time ← now
        continue
    end if
                                    ▷ Block entry logic during cooldown
    if light_on = False and now − exit_time ≤ EXIT_COOLDOWN then
        continue
    end if
                                    ▷ Entry logic
    if light_on = False and now − exit_time > EXIT_COOLDOWN and 0 < last_distance_time − last_motion_time < WINDOW_ENTRY and last_distance > DISTANCE_THRESHOLD then
        brightness ← CALCULATE_BRIGHTNESS(lux)
        if brightness > 0 then
            SET_BULB_BRIGHTNESS(brightness)
            light_on ← True
        end if
        continue
    end if
```

Fig. 10: Pseudocode for Day 5 testing with PIR, mmWave, and light lux sensors (main5.py).

3.5 Testing Procedure

To evaluate the energy consumption patterns under different lighting and context-triggered conditions through sensors, a five-day experiment was conducted in a bedroom to mimic realistic triggers such as ambient lighting changes and movement going in and out of the room. The Philips smart bulb was connected to the THIRDREALITY smart plug, which were both connected to the Raspberry Pi that had Zigbee2MQTT for automation and data collection. The sensors were each used to simulate real-world scenarios including motion detection, human presence, and ambient light responsiveness. In the first four days, the testing period was between 4-8 PM to simulate the typical amount of movement in and out of the room. The testing period took place between 4:19 PM and 8:19 PM specifically for Day 5. This time frame was important for Day 5's testing procedure as this day relied on the changing levels of ambient light in the room. The test setup remained fixed throughout the entire period with the sensors positioned to face in the direction of the door (PIR sensor), toward the center and opposite end of the room (mmWave sensor), and upward to catch ambient lighting (light lux sensor).

Each day consisted of a different test method:

- Day 1: Constant Full Brightness (control test)
  The light bulb was left on at 100% brightness for 4 hours with no sensor control. This acted as a baseline measurement for maximum energy consumption under constant use.
- Day 2: Constant Half Brightness
  The light bulb was left on at 50% brightness for 4 hours with no sensor control. This lets us see how much energy can be saved just by reducing the brightness.
- Day 3: PIR Sensor
  The PIR motion sensor was implemented to detect motion in the room. The bulb turned on at 100% brightness when motion was detected and turned off after 10 seconds and motion was no longer detected. Once the room became too dark, a command was sent to keep the light on for the duration of the test since the PIR sensor would need constant movement to trigger the light on. This simulated a basic motion-based automation setup.

Day	Test Setup	Duration (h)	Average Voltage (V)	Average Current (A)	Peak Power (W)	Average Power (W)	Total Energy Consumed (kWh)
1	Full Brightness	4	120.82	0.072	9.00	7.18	0.037216
2	50% Brightness	4	122.03	0.029	2.50	2.42	0.010028
3	PIR Sensor	4	121.20	0.039	9.20	4.73	0.008961
4	PIR + mmWave Sensor	4	122.55	0.076	9.66	8.64	0.030753
5	PIR + mmWave + Light Lux Sensor	4	120.50	0.036	9.20	3.37	0.007024

Table 1: Charted view of the details for total energy consumed (kWh).

- Day 4: PIR + mmWave Sensors
  The PIR and mmWave sensors were combined to increase detection accuracy and reduce false turn-offs found with the PIR sensor alone (e.g., when a person is in the room but not moving in front of the PIR sensor). The light bulb was activated when the PIR detected motion and the mmWave detected presence, otherwise turned off after 10 seconds of inactivity. The range for the mmWave was set to detect any presence > 800 mm, which covers the distance past the point of entry and exit and signals that the tester is currently staying in the room.
- Day 5: PIR + mmWave + Light Lux Sensors
  Ambient light levels in the room were considered along with motion and presence. The bulb only turned on if motion and continued presence were detected and the lux from the light sensor was detected at certain thresholds and adjusted the brightness of the smart bulb accordingly. After taking a day to get the lux values every 5 minutes from 4-8 PM (baseline values to understand what the lux level is at different times of the day), it was decided to adjust the smart bulb brightness according to the changing lux levels. The light should stay off when lux >= 20.0 lx, and the light should start turning on with brightness rising as the lux decreases and approaches the value where the brightness should hit 100% (in our case that lux is set to 10.0 lx). This method simulates appropriate lighting based on both presence and available daylight.

Energy consumption data was recorded in real time using the smart plug and logged for visualization through a Python script in the Raspberry Pi. This setup allowed accurate tracking of power usage for each method.

Our proposed solution focuses primarily on smart lighting systems since we believe smart lighting is the first step homeowners typically take when delving into the world of implementing IoT devices within the home. As we attempted to introduce more smart devices to test our solution, we noticed that the majority of Zigbee supported devices were out of our budget, so we opted for refining the logic and hardware specifications that go toward energy efficiency in smart lighting systems.

4 Results and Analysis

The table (Table 1) outlines the power, voltage, current, and energy consumption results for each test configuration over a four-hour duration. It summarizes the key electrical metrics measured during each of the five testing days, which are referenced in the analysis that follows.

Day 1 and 2 testing were relatively simple, considering they did not depend on the use of sensors and only relied on brightness levels and the bulb being constantly on.

During day 3 testing using only the PIR motion sensor, it was given that the sensor would not trigger the light bulb to stay on while the tester remained seated and still after turning it on by motion detection (walking by the sensor). The bulb also did not turn on even when ambient lighting decreased significantly. Once the room became too dark, a manual command was sent to turn on the bulb at 7:19 PM. The decision to send the command reflects the limitation of PIR-only detection that does not account for the room getting dark and continual presence in the room. Not much energy was consumed by the PIR since it would only turn on for seconds at a time. It was not until the command was sent to force activate the light bulb that energy consumption started rising.

Day 4 showed unexpected results, but they eventually made sense upon further investigation. It showed the limitations of using both PIR and mmWave sensors. Although more accurate in terms of sensing continual presence in the room compared to day 3's use with only the PIR sensor, the results displayed in the table show that day 4 consumed more energy. This is because the tester's continual presence in the room during long periods kept the light bulb on. The PIR during day 3 would turn on the light bulb when motion was sensed but turn off after 5 seconds. In day 4, motion sensed would get the light bulb ready to turn on or off depending on what the mmWave range read (establishing an entry/exit condition). Using both sensors together was more efficient for the tester in terms of comfortability, especially when the room eventually got dark after sunset and the light bulb would have needed a command manually sent to turn on the bulb permanently. Instead, the tester's continual presence in the room prompted the light bulb to stay on, as intended.

Day 5, the final testing day, consisted of all three sensors working together. The idea was to check for a person's motion with the PIR and if that person's presence was still detected in the room after the PIR sensed motion. The light bulb should only turn on if those two conditions were met and the light lux level was read to be within a certain threshold to adjust the brightness of the bulb. The light started to activate around the same time a manual command would have to be sent to force an activation during previous testing days, which was at around 7:18 PM.

With days 4 and 5, we had some issues positioning the sensors, specifically the PIR and mmWave sensors.

Since the power would go down to 0 Watts (W) when the bulb is in an off state, there was no standby power consumption noticed in any of the tests. The estimated energy consumption would remain the same if the power stayed at 0 W and would only increase if the power increased as well.

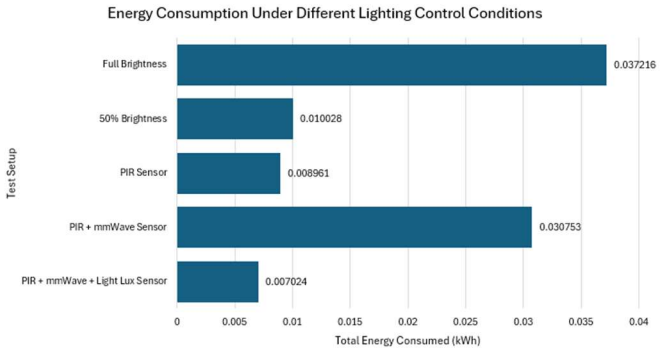

Fig. 11: Charted view of energy consumption under different lighting control conditions.

The figure above (Fig. 11) displays the total energy consumption under the five different lighting control conditions. The full brightness setup resulted in the highest energy consumption at 0.037216 kWh since the bulb remained at maximum brightness for the entire testing period. The 50% brightness setup consumed significantly less energy than the first day but still more energy than sensor-based approaches. This highlights the impact of reduced light output alone. Among the three sensor-based configurations, the PIR and mmWave sensor setup showed a significant spike in energy use (0.030753 kWh) due to its ability to keep the bulb on continuously when the user remained in the room. This offered comfort but at the expense of higher consumption. In contrast, the PIR sensor alone consumed less energy (0.008961 kWh) as it turned off quickly when motion was no longer detected. The most efficient setup, as hypothesized, was the PIR, mmWave, and light lux sensors combined. This not only confirmed motion and continuous presence, but it also adjusted the bulb's brightness based on ambient light and led to the lowest consumption at 0.007024 kWh.

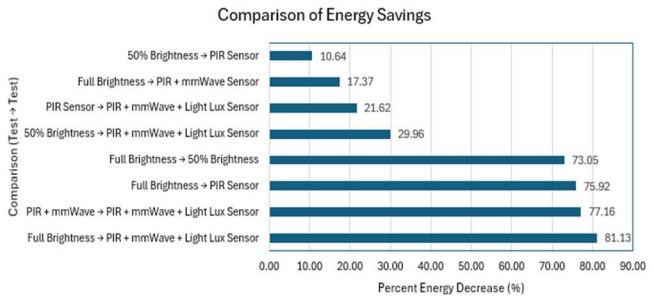

Fig. 12: Bar chart view to show how much each testing method improved over the worst-case baseline.

A bar chart (Fig. 12) was generated to illustrate the performance efficiency of each testing method by transitioning from one lighting control method to another. The comparisons include unique pairs of setups tested in our proposed solution, such as "Full Brightness" from day 1 to "PIR Sensor" to day 3 for example. The most significant energy savings were observed when viewing Full Brightness to PIR, mmWave, and light lux sensor combined, which yielded an 81.1% reduction in energy consumption. This highlights the benefit of combining presence detection with ambient light sensing to the use of motion detection. There are also moderate changes seen in the chart that signify substantial changes, like reducing brightness or implementing a PIR sensor. Altogether, the chart visually reinforces the premise that adding more contextual awareness to lighting control leads to more energy efficiency. One can also see which sensor combinations provide the best balance savings.

5 Conclusion and Future Work

Our results demonstrate that our method reduces wasted energy consumption through contextual triggers from sensors. It also shows a disadvantage of relying only on motion-based triggers for smart lighting, a limitation that should be emphasized. For a method that is commonly used by homeowners with smart lighting systems, it fails to provide adequate lighting according to contextual input such as less ambient light or continual presence that a motion sensor cannot detect. This reinforces the importance of incorporating more sensor integration, such as the mmWave and light sensors, for more accurate and responsive lighting control.

Future work can find ways to connect the sensors to the lighting system itself, which would eliminate the use of needing the Raspberry Pi as a central controller. Due to constraints with materials and budget, future work could also expand energy monitoring and reduce wasted consumption beyond lighting systems to include other smart home appliances. This would enhance whole-room or home energy optimization, allowing for a more comprehensive energy management strategy while maintaining user comfort. It would also validate the scalability and flexibility of the proposed sensor-automation model.

6 References

[1] M. Roscia, V. Dancu and G. C. Lazaroiu, "Smart Home Survey Analysis," 2023 IEEE International Smart Cities Conference (ISC2), Bucharest, Romania, 2023, pp. 1-5.

[2] M. I. Abdullah, D. Roobashini and M. H. Alkawaz, "Active Monitoring of Energy Utilization in Smart Home Appliances," 2021 IEEE 11th IEEE Symposium on Computer Applications & Industrial Electronics (ISCAIE), Penang, Malaysia, 2021, pp. 245-249.

[3] L. S, H. R, D. Jayalakshmi, V. R. Vimal and L. Kanthan Narayanan, "IoT-Driven Energy Consumption Optimization in Smart Homes," 2024 International Conference on Trends in Quantum Computing and Emerging Business Technologies, Pune, India, 2024, pp. 1-5.

[4] W. Wang, J. Su, Z. Hicks and B. Campbell, "The Standby Energy of Smart Devices: Problems, Progress, & Potential," 2020 IEEE/ACM Fifth International Conference on Internet-of-Things Design and Implementation (IoTDI), Sydney, NSW, Australia, 2020, pp. 164-175.

[5] O. Ayan and B. Turkay, "IoT-Based Energy Efficiency in Smart Homes by Smart Lighting Solutions," 2020 21st International Symposium on Electrical Apparatus & Technologies (SIELA), Bourgas, Bulgaria, 2020, pp. 1-5.

[6] S. K. N, J. S, K. S and P. S, "Implementation of IoT Based Smart Home Automation and Energy Conservation," 2024 5th International Conference on Data Intelligence and Cognitive Informatics (ICDICI), Tirunelveli, India, 2024, pp. 153-157.

[7] H. Lee, W. -K. Park and I. -W. Lee, "A Home Energy Management System for Energy-Efficient Smart Homes," 2014 International Conference on Computational Science and Computational Intelligence, Las Vegas, NV, USA, 2014, pp. 142-145.

[8] O. M. Longe, K. Ouahada, S. Rimer, H. Zhu and H. C. Ferreira, "Effective energy consumption scheduling in smart homes," AFRICON 2015, Addis Ababa, Ethiopia, 2015, pp. 1-5.

[9] P. M R and B. Bhowmik, "Development of IoT-Based Smart Home Application with Energy Management," 2023 International Conference on Sustainable Communication Networks and Application (ICSCNA), Theni, India, 2023, pp. 367-373.

[10] N. Goel, S. Bhatia and P. Rana, "Secure Smart Devices Control with Low Power Consumption," 2023 3rd International Conference on Advancement in Electronics & Communication Engineering (AECE), GHAZIABAD, India, 2023, pp. 83-87.

[11] M. V. Rafliana Roostandi, T. A. Nathaniel, D. Qinthara and S. S. T. Boby, "Smart Light Control Using Thermal Sensor As Human Presence Detection," 2021 4th International Conference on Information and Communications Technology (ICOIACT), Yogyakarta, Indonesia, 2021, pp. 75-79.

[12] M. K. Hossain, "Smart & Scalable Lighting System Using Motion Sensors, Light Dependent Resistors and ESP8266 Controller Board," 2024 IEEE International Conference on Advanced Systems and Emergent Technologies (IC_ASET), Hammamet, Tunisia, 2024, pp. 1-6.

[13] B. K. Barman, S. N. Yadav, S. Kumar and S. Gope, "IOT Based Smart Energy Meter for Efficient Energy Utilization in Smart Grid," 2018 2nd International Conference on Power, Energy and Environment: Towards Smart Technology (ICEPE), Shillong, India, 2018, pp. 1-5.

[14] V. T, D. K, H. M and P. M, "Smart Energy Meter Based on IoT Utilizing Raspberry Pi and Arduino," 2023 Third International Conference on Ubiquitous Computing and Intelligent Information Systems (ICUIS), Gobichettipalayam, India, 2023, pp. 91-96.

[15] M. O. Agyeman, Z. Al-Waisi and I. Hoxha, "Design and Implementation of an IoT-Based Energy Monitoring System for Managing Smart Homes," 2019 Fourth International Conference on Fog and Mobile Edge Computing (FMEC), Rome, Italy, 2019, pp. 253-258.